

Haval H6 — Offline Recon Command Reference



Haval H6 — Offline Recon Command Reference

- ✓ 0. Pre-flight checklist — DO THIS BEFORE YOU LOSE INTERNET
 - 🔧 1. Set up your research workspace
 - 🔧 2. Connect to the car's WiFi & confirm you're on it
 - 🔧 3. Discover the head unit's IP address
 - 🔧 4. Scan the head unit for open ports
 - 🔧 5. Connect via ADB over WiFi
 - 🔧 6. Device identity & hardware
 - 🔧 7. System properties — the hidden config database
 - 🔧 8. Installed packages & apps (pm)
 - 🔧 9. Settings databases (system config stores)
 - 🔧 10. Logs — the ground truth (logcat)
 - 🔧 11. System services & state (dumpsys / service)
 - 🔧 12. Filesystem exploration
 - 🔧 13. Pull APKs for offline analysis
 - 🔧 14. Screenshots, recording & UI dumps
 - 🔧 15. Input automation (tap / swipe / keys)
 - 🔧 16. Launch hidden activities & settings screens
 - 🔧 17. Networking — inside the head unit
 - 🔧 18. Car-specific / CAN bus / vehicle services
 - 🔧 19. RISKY / POTENTIALLY DESTRUCTIVE — read carefully
 - 🔧 20. Save everything before you disconnect
 - 🔧 21. Troubleshooting quick reference
 - 🔧 22. CAN Bus Sniffing — two routes
 - 🔧 23. Custom OBD-II App — Roadmap
 - 🔧 Appendix A — keycode reference
 - 🔧 Appendix B — minimum viable first session
 - 🔧 References



Haval H6 — Offline Recon Command Reference

Complete command reference for probing your 2021 Haval H6 head unit from a MacBook connected to the car's WiFi. **Assumes NO internet access.**

⚠ These commands operate on **your own vehicle**. Several can brick the head unit, disable features, or void warranty. Read the ⚠ markers before running anything. When in doubt, **read-only commands first** (getprop, dumpsys, ls, pm list).



0. Pre-flight checklist — DO THIS BEFORE YOU LOSE INTERNET

Everything you need installed ahead of time (your Mac needs internet for these):

```
# Android platform tools (adb, fastboot) – REQUIRED
brew install --cask android-platform-tools

# Network scanners – HIGHLY recommended
brew install nmap
brew install arp-scan
brew install netcat          # (macOS already ships `nc`, but this is newer)

# APK reverse engineering (to analyze apps you pull off the head unit)
brew install jadx           # NOTE: jadx is a formula, NOT a cask (no --cask)
brew install apkttool

# Optional extras
brew install mtr             # network path tracing
brew install wireshark       # packet capture (has a GUI)
```

Verify they run:

```
adb version
nmap --version
arp-scan --version
nc -h >/dev/null 2>&1; echo "nc OK"
```

ping, arp, route, ifconfig, ipconfig, tcpdump, curl, nc are all **built into macOS** — no install needed.

Copy this file somewhere you can open offline (it's already on your disk — don't delete it, and maybe email it to yourself as backup).

1. Set up your research workspace

```
mkdir -p ~/haval-research
cd ~/haval-research
```

Save every command's output (never lose a probe again):

```
# shell function: run a command and tee its output to a timestamped file
function hav() { mkdir -p ~/haval-research; local f=~/.haval-research/$(date +%Y%m%d-%H%M%S)_$(echo "$*" | sed 's/[^a-zA-Z0-9]/_/g' | cut -c1-40).txt; "$@" 2>&1 | tee "$f"; }

# usage:
hav adb shell getprop          # dumps props AND saves them
hav nmap -p- 192.168.43.1      # dumps scan AND saves it
```

Define the head unit IP once (change to whatever you discover in §3):

```
export HU=192.168.43.1          # tip: the car hotspot gateway is often the head unit itself
export ADB="adb connect $HU:5555"
```



2. Connect to the car's WiFi & confirm you're on it

```
# Show your WiFi interface(s) and status
networksetup -listallhardwareports

# Show your current IP on en0 (your WiFi adapter)
ipconfig getifaddr en0

# Show ALL interfaces with IPv4 addresses
ifconfig | grep -A 2 'inet '
ifconfig en0

# Show the default gateway (this is almost always the head unit / hotspot host)
route -n get default
route -n get default | grep gateway

# Confirm you can reach the network (ping the gateway)
ping -c 4 192.168.43.1
```

Android hotspot subnet note: Android-based hotspots (which a Haval head unit effectively is) almost always use **192.168.43.0/24**, gateway **192.168.43.1**, DHCP range 192.168.43.100–254. If your Mac's IP starts with 192.168.43.x, the head unit is very likely **192.168.43.1**.



3. Discover the head unit's IP address

```
# Easiest: the default gateway (see §2)
route -n get default | grep gateway

# List everything in the ARP table (devices your Mac has talked to)
arp -a

# Full subnet sweep (built into macOS, slow but works)
# First figure out your subnet from your IP, e.g. 192.168.43.10 => 192.168.43.0/24
ping -c 2 192.168.43.1 && ping -c 2 192.168.43.254

# Fast live-host discovery (needs nmap)
nmap -sn 192.168.43.0/24

# Live-host discovery (needs arp-scan; run with sudo for best results)
sudo arp-scan --localnet
sudo arp-scan 192.168.43.0/24

# Show neighbors again after a scan – hosts that responded show up here
arp -a
```

What you're looking for: a host that isn't your Mac — typically 192.168.43.1 (gateway). MAC vendor prefixes like 0C:FC:83 or 48:AA:5D (Great Wall / automotive) are strong hints.

4. Scan the head unit for open ports

```
# Common car-infotainment ports, fast (needs nmap)
nmap -T4 -p 21,22,23,53,80,443,888,2000,2177,5000,5555,6000,7000,8000,8080,8888,9000 $HU

# Full port scan (65535 ports – takes a few minutes, needs nmap)
nmap -p- -T4 --min-rate 1000 $HU

# Service/version detection on discovered ports
nmap -sV -p 5555,22,80 $HU

# Test a single port manually (nc is built into macOS)
nc -zv $HU 5555
nc -zv $HU 22
nc -zv -w 3 $HU 5555
```

Ports you care about most:

Port	Service	Why it matters
5555	ADB	Full adb shell access — the big one
22	SSH	Root shell if reachable
23	Telnet	Legacy unauthenticated shell
80/8080	HTTP	Web config / API endpoints
7000–7001	GWM/Gateway	Vehicle telematics
8000	OTA/media	Update services
2177	Diagnostics	Some OEM tools

```
# Grab the banner/HTTP page from any web service you find
curl -v http://$HU/
curl -v http://$HU:8080/
curl -v -X OPTIONS http://$HU/

# Raw banner grab for any port
printf 'GET / HTTP/1.1\r\nHost: %s\r\n\r\n' "$HU" | nc -w 3 $HU 80
```

5. Connect via ADB over WiFi

```
# Start fresh
adb kill-server
adb start-server

# Try to connect (the car usually answers on 5555)
adb connect $HU:5555

# List devices – look for "device" (not "offline" / "unauthorized")
adb devices
adb devices -l

# If the first connect hangs or says "offline", retry
adb disconnect
adb connect $HU:5555
```

Authorization: the head unit will pop up an **“Allow USB debugging?”** RSA fingerprint dialog on its screen the first time. You MUST tap “Allow” (and optionally “Always allow from this computer”) on the car’s display or it will stay **unauthorized**.

If no dialog appears: ADB may be disabled on the unit, or it’s not listening. That doesn’t necessarily mean game over — see §20 troubleshooting.

Now test the shell:

```
adb -s $HU:5555 shell id
adb -s $HU:5555 shell whoami
adb -s $HU:5555 shell echo connected
```



6. Device identity & hardware

```
# Everything at once (this is the single most useful dump – save it)
adb shell getprop > props.txt

# Quick identity
adb shell getprop ro.product.model
adb shell getprop ro.product.brand
adb shell getprop ro.product.name
adb shell getprop ro.product.device
adb shell getprop ro.build.version.release      # Android version
adb shell getprop ro.build.version.sdk
adb shell getprop ro.build.version.security_patch
adb shell getprop ro.board.platform             # SoC (rockchip? mtk? qualcomm?)
adb shell getprop ro.product.cpu.abi
adb shell getprop ro.hardware
adb shell getprop ro.serialno
adb shell getprop ro.build.fingerprint
adb shell getprop ro.debuggable                 # 1 = adb enabled as root-capable
adb shell getprop ro.secure                     # 0 = adbd runs as root
adb shell getprop ro.build.type                 # user / userdebug / eng
adb shell getprop ro.build.tags
adb shell getprop ro.boot.slot_suffix           # A/B slots
adb shell getprop ro.carrier
adb shell getprop ro.bootmode

# Kernel / OS info
adb shell uname -a
adb shell cat /proc/version
adb shell cat /proc/cpuinfo
adb shell cat /proc/meminfo
adb shell cat /proc/uptime

# Storage & memory
adb shell df -h
adb shell free -m

# Are we root?
adb shell id
adb shell ls -la /data
adb shell su -c id
adb shell which su

# What's mounted and where
adb shell mount
adb shell cat /proc/mounts
```

ro.debuggable=1 + ro.secure=0 would be jackpot (adbd already root). Almost certainly you'll see **ro.secure=1**, meaning read-only shell.



7. System properties — the hidden config database

```
# Full dump (save it)
adb shell getprop > props.txt

# Grep for everything vendor/GWM specific
adb shell getprop | grep -i gwm
adb shell getprop | grep -i haval
adb shell getprop | grep -i persist
adb shell getprop | grep -i vendor
adb shell getprop | grep -i config
adb shell getprop | grep -i feature
adb shell getprop | grep -i version
adb shell getprop | grep -i adb
adb shell getprop | grep -i usb
adb shell getprop | grep -i can
adb shell getprop | grep -i car
adb shell getprop | grep -i wifi

# Everything with "cfg" (config flags – the interesting ones)
adb shell getprop | grep -i cfg

# Individual property lookups
adb shell getprop persist.vendor.gwm.cfg.head.up.display
adb shell getprop persist.vendor.gwm.cfg.auto.high.beam
adb shell getprop persist.vendor.gwm.cfg.adas.enable
```

⚠ WRITE ACCESS IS DANGEROUS

setprop on a locked-down production unit usually fails, but if it doesn't:

```
adb shell setprop persist.vendor.gwm.cfg.head.up.display 1 # ⚠ could do nothing OR
break things
```

Read-only exploration first. **Do not blind-set vendor props.** Some are sanity-checked against CAN bus hardware and forcing them can throw the car into fault states.

8. Installed packages & apps (pm)

```
# All packages
adb shell pm list packages
adb shell pm list packages | grep -i gwm
adb shell pm list packages | grep -i haval
adb shell pm list packages | grep -i car
adb shell pm list packages | grep -i can
adb shell pm list packages | grep -i vehicle
adb shell pm list packages | grep -i appstore
adb shell pm list packages | grep -i settings

# Third-party / OEM apps (these are the interesting ones)
adb shell pm list packages -3
adb shell pm list packages -3 -f          # ...with APK file paths

# Disabled / enabled packages
adb shell pm list packages -d            # disabled
adb shell pm list packages -e            # enabled

# System package paths
adb shell pm list packages -f | grep -i gwm

# Hardware & software features the build claims
adb shell pm list features
adb shell pm list libraries

# Details of a specific app (activities, services, permissions, flags)
adb shell dumpsys package com.gwm.xxx

# Find the launchable activity of any app (for hidden launcher entries)
adb shell cmd package resolve-activity --brief -a android.intent.action.MAIN -c
    android.intent.category.LAUNCHER
adb shell cmd package resolve-activity --brief -a android.intent.action.MAIN -c
    android.intent.category.LAUNCHER | grep -v '^$'
```

Enabling/disabling apps

You can toggle apps but **do NOT disable system apps** (`pm disable`) on a vehicle — things like the instrument cluster, HVAC, or CAN services could disappear.

```
adb shell pm list packages -d          # see what's already disabled
# adb shell pm disable-user <pkg>      # only try on an obviously-safe app
```


9. Settings databases (system config stores)

```
# Dump all three settings namespaces (read-only, safe)
adb shell settings list global
adb shell settings list system
adb shell settings list secure

# Individual lookups
adb shell settings get global adb_enabled
adb shell settings get global development_settings_enabled
adb shell settings get secure android_id
adb shell settings get system volume_music

# Backup route via dumpsys
adb shell dumpsys settings
```

Writing settings

```
adb shell settings put global adb_enabled 1 #  may not persist / may be blocked
```

Same caution as `setprop`: on production units these are typically locked, and force-writing can cause instability. Prefer reading.

10. Logs — the ground truth (logcat)

```
# Dump the entire current log (save it)
adb shell logcat -d > logcat.txt
adb logcat -d > logcat.txt # same, from the Mac side

# Dump all log buffers (main, system, crash, radio, events, kernel)
adb shell logcat -d -b all > logcat_all.txt

# Watch live (Ctrl+C to stop)
adb logcat

# Filter for the interesting stuff
adb logcat -d | grep -iE 'gwm|haval|canbus|can_bus|car_service|vehicle'
adb logcat -d | grep -iE 'adb|daemon|5555'
adb logcat -d | grep -iE 'error|exception|fatal' | tail -100

# Per-buffer filters
adb logcat -d -b events | grep -iE 'car|vehicle|wifi'
adb logcat -d -b crash
adb logcat -d -b radio | grep -iE 'can|canbus'

# By tag
adb logcat -d -s CarService:E
adb logcat -d -s AndroidRuntime:E

# Kernel messages
adb shell dmesg | grep -iE 'gwm|can|wifi|usb'
```

11. System services & state (dumpsys / service)

```
# List every running binder service
adb shell service list

# Dump a specific service
adb shell dumpsys wifi
adb shell dumpsys connectivity
adb shell dumpsys netstats
adb shell dumpsys battery
adb shell dumpsys audio
adb shell dumpsys media_session
adb shell dumpsys package
adb shell dumpsys activity activities
adb shell dumpsys activity top          # current screen
adb shell dumpsys window
adb shell dumpsys input
adb shell dumpsys display
adb shell dumpsys usb
adb shell dumpsys mount
adb shell dumpsys alarm
adb shell dumpsys meminfo
adb shell dumpsys cpuinfo
adb shell dumpsys gfxinfo
adb shell dumpsys content

# Car / vehicle services (Android Automotive / OEM stacks)
adb shell dumpsys car
adb shell dumpsys car_service
adb shell dumpsys canbus
adb shell dumpsys vehicle
adb shell dumpsys systemuiautomation

# Everything into one file (huge but complete)
adb shell dumpsys > dumpsys_all.txt

# Call an arbitrary binder service method directly (advanced)
adb shell service call connectivity 1
adb shell service list | grep -iE 'car|can|vehicle'
```

12. Filesystem exploration

```
# Top-level layout
adb shell ls -la /
adb shell ls -la /system /vendor /persist /data /storage /sdcard

# App dirs
adb shell ls -la /system/app
adb shell ls -la /system/priv-app
adb shell ls -la /system/vendor/app
adb shell ls -la /vendor/app
adb shell ls -la /data/app           # may be restricted without root
adb shell ls -la /data/data         # may be restricted without root

# Find anything GWM-named
adb shell ls -R /system | grep -iE 'gwm|haval|auto|car' | head -100
adb shell ls -R /vendor | grep -iE 'gwm|haval|auto|car' | head -100

# Key config files
adb shell cat /system/build.prop
adb shell cat /vendor/build.prop
adb shell cat /system/etc/hosts
adb shell cat /system/etc/*.conf 2>/dev/null
adb shell cat /proc/cmdline
adb shell cat /proc/partitions

# What can you actually write to? (look for "rw" mounts)
adb shell mount | grep rw

# Storage layout
adb shell df -h
adb shell ls -la /persist
adb shell cat /persist/build.prop 2>/dev/null
```

`/persist` is where cars often keep feature flags / VIN / config that survives OTA. Read it if accessible — but **never write to it** without full certainty.

13. Pull APKs for offline analysis

```
# Find the full path of an app's APK
adb shell pm path com.gwm.xxx

# Pull one APK
adb pull /system/app/GWMAApp/GWMAApp.apk .
adb pull /system/priv-app/CarService/CarService.apk .

# Pull ALL system + priv apps (can be slow / large)
adb pull /system/app ./system_app
adb pull /system/priv-app ./system_priv_app

# Save the package-path map too
adb shell pm list packages -f > packages_with_paths.txt
```

Analyze offline with the tools from §0:

```
jadx CarService.apk          # decompile to Java source
apktool d CarService.apk     # decode resources & smali
# jadx-gui for a GUI
```

What to look for inside APKs: - Strings: `grep -r "setprop" .`, `grep -ri "gwm.cfg" .` - Hidden activities / broadcast receivers (manifest) - Hardcoded IPs/ports for gateway/MQTT/CAN services - Feature flag names (compare against `getprop` | `grep cfg`)

14. Screenshots, recording & UI dumps

```
# Screenshot the head unit screen
adb shell screencap -p /sdcard/screen.png
adb pull /sdcard/screen.png ./screen.png

# Or straight to the Mac without touching the unit's storage
adb exec-out screencap -p > screen.png
sips -g pixelWidth -g pixelHeight screen.png    # check dims

# Screen resolution / density
adb shell wm size
adb shell wm density
adb shell dumpsys window displays

# Record video (up to 3 min default; Ctrl+C to stop)
adb shell screenrecord --size 1280x720 --bit-rate 4000000 /sdcard/rec.mp4
# ^ then Ctrl+C, then:
adb pull /sdcard/rec.mp4 .

# UI hierarchy XML – shows every on-screen element with bounds & text
adb shell uiautomator dump /sdcard/ui.xml
adb pull /sdcard/ui.xml ./ui.xml
# inspect with:
xmllint --format ui.xml
# find tap-able coords:
xmllint --xpath '//node[@text!=""]/@bounds' ui.xml
```

👉 15. Input automation (tap / swipe / keys)

```
# Screen coordinate taps (get coords from the uiautomator dump)
```

```
adb shell input tap 600 800
adb shell input swipe 300 800 900 800 300
adb shell input keyevent 3      # HOME
adb shell input keyevent 4      # BACK
adb shell input keyevent 66     # ENTER
adb shell input keyevent 82     # MENU
adb shell input keyevent 127    # SEARCH
adb shell input keyevent 24     # VOLUME UP
adb shell input keyevent 25     # VOLUME DOWN
```

```
# Type text (useful for entering WiFi/account creds)
```

```
adb shell input text "hello"
```

```
# Turn screen on/off
```

```
adb shell input keyevent 26
adb shell input keyevent 223    # SLEEP
```

🖥️ 16. Launch hidden activities & settings screens

```
# Stock Android settings screens (safe)
```

```
adb shell am start -a android.settings.SETTINGS
adb shell am start -a android.settings.WIFI_SETTINGS
adb shell am start -a android.settings.BLUETOOTH_SETTINGS
adb shell am start -a android.settings.APPLICATION_SETTINGS
adb shell am start -a android.settings.APPLICATION_DEVELOPMENT_SETTINGS # developer
options
adb shell am start -a android.settings.ADB_WIRELESS_SETTINGS
adb shell am start -a android.settings.DATE_SETTINGS
adb shell am start -a android.settings.SOUND_SETTINGS
adb shell am start -a android.settings.USB_MTP_SETTINGS
```

```
# Launch any app by explicit component
```

```
adb shell am start -n com.android.settings/.Settings
adb shell am start -n com.gwm.xxx/.HiddenActivity
```

```
# Enumerate all activities of a package (find hidden ones)
```

```
adb shell dumpsys package com.gwm.xxx | grep -A 500 'Activity Resolver' | grep -E
'^\s+[0-9a-f]+ '
```

```
# Send a broadcast (advanced)
```

```
adb shell am broadcast -a android.intent.action.SCREEN_ON
```

⚠️ Developer options & hidden settings

Opening Developer Options and toggling things like “Wireless debugging” or “OEM unlock” may require a password, may reboot the unit, or may enable additional access. Use cautiously.

17. Networking — inside the head unit

```
# The head unit's own addresses
adb shell ip addr
adb shell ip route
adb shell ifconfig
adb shell ip neigh                # its ARP table (who it talks to)

# What services is IT listening on?
adb shell netstat -tulpn
adb shell cat /proc/net/tcp
adb shell cat /proc/net/udp

# Test connectivity from the head unit (to other cars' / phones' services)
adb shell ping -c 3 8.8.8.8        # only if unit has its own internet
adb shell ping -c 3 192.168.43.100 # its DHCP client (your Mac)

# WiFi state
adb shell dumpsys wifi
adb shell cmd wifi status
adb shell wpa_cli status
adb shell wpa_cli list_networks    # saved networks (incl. credentials!)
```

18. Car-specific / CAN bus / vehicle services

```
# Vehicle-level services
adb shell dumpsys car
adb shell dumpsys car_service
adb shell service list | grep -iE 'car|can|vehicle|diag'

# CAN interfaces & devices
adb shell ls /dev | grep -i can
adb shell ls -la /dev/can*
adb shell cat /proc/net/can 2>/dev/null
adb shell ip -details link show can0 2>/dev/null

# Real-time input/event stream (touch, buttons, steering)
adb shell getevent
adb shell getevent -c 50          # capture 50 events then stop
```

CAN bus

Directly reading `/dev/can*` requires root and even then raw CAN injection on a live bus can disable safety systems. **Do not** read from or write to CAN devices unless you fully understand the bus topology and have isolated/disconnected the vehicle.

19. RISKY / POTENTIALLY DESTRUCTIVE — read carefully

These can **brick the unit, blank the cluster, or require dealer intervention**. Only run them if you accept that risk:

```

# Reboot the head unit (safe-ish: it's just a reboot)
adb reboot
adb shell reboot

# ⚠ Recovery / bootloader – CAN BRICK OR LOCK YOU OUT
adb reboot recovery      # ⚠ may be unmodified/absent on OEM builds
adb reboot bootloader    # high risk of brick on OEM head units

# ⚠ Wipe everything on the unit
adb shell pm clear com.gwm.xxx      # clears ONE app's data
adb shell wm size reset
adb shell settings put global adb_enabled 0  # disables adb!
# factory reset = only via the unit's own menu, NOT recommended

# ⚠ Rogue shell attempts (some fail harmlessly, some get you fired at – root only)
adb shell su -c 'cat /dev/can0'
adb shell mount -o rw,remount /system  # almost certainly denied

```

Golden rule: one change at a time, note the exact command, screenshot the before-state, and be ready that a reboot may revert or may not.



20. Save everything before you disconnect

```

cd ~/haval-research
mkdir -p wifi_scan apks props logs dumpsys ui screenshots

# Collate what you grabbed:
cp ~/haval-research/*.txt wifi_scan/ 2>/dev/null
adb shell getprop > props/full_props.txt
adb shell pm list packages -f > packages_with_paths.txt
adb shell logcat -d > logs/logcat.txt
adb shell dumpsys > dumpsys/full_dumpsys.txt
adb shell settings list global > settings_global.txt
adb shell settings list system > settings_system.txt
adb shell settings list secure > settings_secure.txt

# Copy the whole research folder into your repo
cp -r ~/haval-research/* /path/to/haval_h6_Infotainment/

```

21. Troubleshooting quick reference

Symptom	Fix
<code>adb devices</code> shows nothing	Confirm WiFi link: <code>ping \$HU</code> ; try <code>adb connect \$HU:5555</code> again
Device shows <code>offline</code>	<code>adb disconnect</code> then reconnect; sometimes needs <code>adb kill-server</code>
Device shows <code>unauthorized</code>	Accept the RSA dialog on the car's screen
Connect times out	Port 5555 closed — re-scan ports (\$4); try 5556–5560, or check if unit has a different ADB port
“no devices/emulators found”	<code>adb start-server</code> first; ensure you're still on the car's WiFi
ADB connects but shell is empty	Try <code>adb -s \$HU:5555 shell getprop ro.build.version.release</code>
Can't find the head unit IP	Use <code>route -n get default gateway</code> ; <code>sudo arp-scan -- localnet</code> ; power-cycle car WiFi
Head unit screen won't allow dialog	Try developer options via <code>adb shell am start -a android.settings.APPLICATION_DEVELOPMENT_SETTINGS</code>
Everything read-only	Expected on production builds — fall back to <code>getprop</code> / <code>dumpsys</code> / <code>logcat</code> / APK analysis

22. CAN Bus Sniffing — two routes

Your Mac has no CAN port, so you need a hardware interface. Two routes:

	Route A — external sniffer	Route B — head unit software
Where	OBD2 port (or tap head-unit harness)	Inside the head unit, if rooted
What you see	Whatever the gateway forwards on the OBD bus	Exactly what the head unit sees (infotainment CAN: cluster, ADAS, 4x4, HVAC)
Cost	~\$10–30 + Raspberry Pi	\$0 (software)
Feasibility	Works today	Depends on ADB / root

For the custom-app goal, **Route B is the prize** (gear, drive mode, torque split live there), but Route A is what you can do right now.

Route A — the classic sniffing rig

Hardware:

- **Raspberry Pi** (any model) + **MCP2515 SPI CAN module** (~\$8) — the standard cheap rig
- OR a **PCAN-USB** / **Kvaser** dongle (has macOS drivers, but €100+; Linux-first tooling)
- ⚠️ An **ELM327 WiFi OBD adapter cannot sniff** — it is a diagnostic master, not a passive listener

Wiring — OBD2 (DLC) connector, port facing you:


```
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
```

- **Pin 6 = CAN-H, Pin 14 = CAN-L** (high-speed CAN, ISO 15765-4)
- **Pin 4/5 = ground** (chassis / signal ground) — required
- Power the MCP2515 module's transceiver from 5V; ground with the car's ground
- Modern cars run **500 kbps** (some 250 kbps). Add a 120 Ω terminator only if the bus lacks one.

Software — Linux/SocketCAN (needs internet ONLY for this one-time install):

```
sudo apt install can-utils

# Bring up the interface (MCP2515 on SPI0)
sudo ip link set can0 up type can bitrate 500000

# Watch everything (Ctrl+C to stop)
candump can0

# Log to file with timestamps (main capture command)
candump -l can0 # writes candump-*.log
candump can0 -T <id> -L > capture.log # filter single ID, line mode

# Live "what changed" view – highlights byte diffs per ID
cansniffer can0

# Bus load
canbusload can0

# Inject test frames (ONLY on a bench / known bus – see §19)
cansend can0 123#DEADBEEF
```

Analysis on your Mac (offline):

- **SavvyCAN** (free GUI, runs on macOS) — purpose-built for CAN reverse engineering: open candump logs, graph bytes, auto-diff frames. This is the tool you want.
- **Wireshark** — reads candump logs; can capture SocketCAN on Linux.
- Copy logs Pi → Mac via USB/SD card — no internet needed.

Route B — sniffing from inside the head unit

```
adb shell ls /dev/can* # does the unit expose CAN devices?
adb shell ip -details link show can0 # does it have a CAN interface?
adb shell dumpsys car_service # vendor vehicle service – read-only goldmine
adb shell dumpsys | grep -iE 'can|vehicle|cluster'
adb shell logcat -d | grep -iE 'can|vehicle'
```

The head unit already decodes all infotainment-CAN messages for its cluster/ADAS displays. If `can-utils` exist on it you can `candump` directly; otherwise the vendor vehicle service (`dumpsys car_service`) often exposes decoded values — easier than raw frames.

Reverse-engineering workflow (both routes)

1. **Baseline log** — 2–3 minutes idle (ignition on, engine off, then engine on).
2. **Trigger one action** — press one button / change one setting / toggle high beams.
3. **Diff the logs** — the frame whose ID and bytes changed is your signal.

4. **Decode it** — repeat while varying the physical value (e.g. open a window 10 cm at a time, watch the byte change).
5. **Graph bytes in SavvyCAN** to find linear/sensor encodings (how you'd map gear position, torque split, ADAS states).

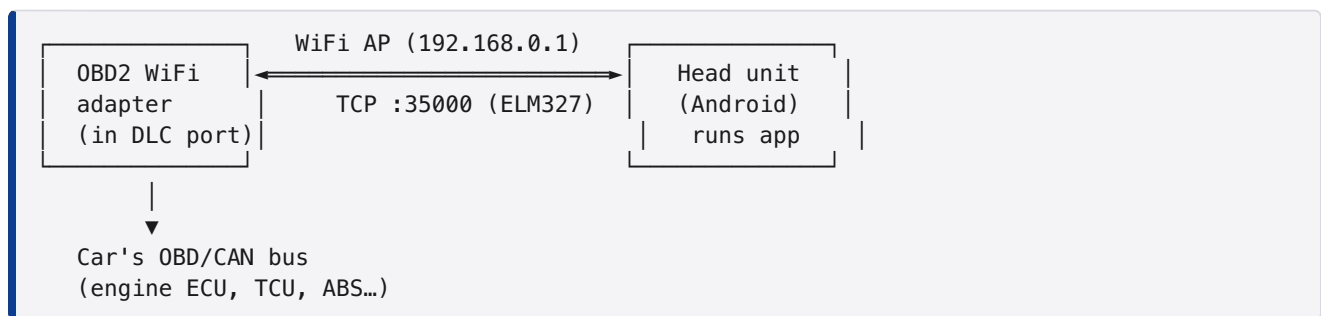
Reality checks & safety

- **OBD port ≠ head unit bus.** The DLC port exposes the *powertrain* CAN; the head unit sits on the *infotainment* CAN behind the dash. The gateway decides what crosses. For gear/4x4/ADAS you'll likely need to **tap the head unit's harness** (the CAN twisted pair behind the unit) rather than the OBD port.
- **Gateway filtering:** modern cars block many diag messages at the gateway — expect only what the gateway allows.
- **Start read-only.** Never inject frames on a live vehicle bus until you know the IDs — CAN injection has triggered wipers, doors, and even steering on some platforms.

23. Custom OBD-II App — Roadmap

Goal: a custom Android app on the head unit that (1) shows all available vehicle data, then (2) renders gauges & graphs. **Very doable** — it's what Torque Pro / Car Scanner do — but there is one hard gate first.

Data flow



- A **WiFi OBD2 adapter** (ELM327-based; OBDLink MX+ is far more reliable than generic clones) creates its own hotspot — typically `192.168.0.10`, TCP **port 35000**, speaking ELM327 AT commands + OBD PIDs.
- The app connects with a plain TCP socket and queries PIDs.

⚠️ WiFi radio conflict

The head unit has **one WiFi radio**, but both ADB-over-WiFi (§5) and OBD-over-WiFi want it. **Sequence it:** install the APK first (while the unit hosts its hotspot for `adb connect`), *then* switch the unit to join the OBD adapter's WiFi. Or use USB ADB for installs if a debug-capable USB port exists.

What data you'll actually get

Standard OBD PIDs (Mode 01) — the easy 70%:

PID	Meaning	PID	Meaning
010C	RPM	012F	Fuel level
010D	Road speed	0142	Battery voltage
0105	Coolant temp	0110	MAF
010F	Intake temp	03	DTCs (Mode 03)

```
# Raw ELM327 exchange
# App → "010C\r"      request RPM
# Unit → "41 0C 0B B8\r"  RPM = (0x0B*256+0xB8)/4 = 754
```

NOT on standard OBD — the “everything else” on a 2.0T 4x4: gear position, drive mode, rear-axle torque split, transfer-case state, ADAS/ACC state, HUD data, range. These come from **GWM’s private CAN messages** (§22), not OBD PIDs — the gateway may block them too. The head unit *already knows* most of it from its own CAN, so the richest path is the head unit’s vendor services, not OBD.

Phase 1 — “see all info first”

1. **Probe supported PIDs:** request 0100, 0120, 0140 ... — the response bitmask lists which PIDs the ECU supports.
2. **Poll all supported PIDs** at ~1–2 Hz; log everything to **SQLite (Room)** / CSV.
3. **Render a live table:** PID → name → value → unit, with a supported/unsupported flag per PID.

Phase 2 — gauges & graphs

- **Gauges:** custom Canvas widgets or a speedometer lib — arc gauge for RPM, linear gauge for coolant temp, dial for speed.
- **Graphs:** **MPAndroidChart** — the standard for real-time line graphs.
- **Diagnostics:** DTC read/clear, freeze-frame viewer, CSV export.

Recommended stack

- Kotlin + Jetpack Compose (or classic Views — head units run old Android, so target **API 26-ish**, not newest)
- **pires/obd-java-api** (GitHub) — battle-tested ELM327/OBD library; avoid hand-rolling the protocol
- MPAndroidChart for graphs, Room for logging

The real bottleneck — installing the APK

- `adb shell pm install` on a production, non-rooted unit is frequently **blocked** (INSTALL_FAILED_USER_RESTRICTED).
- If ADB-over-WiFi works and `pm install` succeeds → you’re in business.
- **Fallback:** run the app on a separate Android phone/tablet mounted in the cabin, on the same OBD WiFi — same app, same data, minus the factory screen.

Recommended sequence

1. **Validate cheaply first:** Torque Pro / Car Scanner on your phone → OBD adapter’s WiFi → check which PIDs your H6 exposes. Generic ELM327 clones fail on some GWM cars; this test costs nothing.
2. Get ADB-over-WiFi working and test whether `adb push + pm install` succeeds (§5).
3. If yes → build Phase 1 (PID probe + live table), sideload, iterate.
4. Phase 2 (gauges/graphs) once the data layer is proven.

Appendix A — keycode reference

Key	Code		Key	Code
HOME	3		BACK	4
CALL	5		ENDCALL	6
VOLUME_UP	24		VOLUME_DOWN	25
POWER	26		CAMERA	27
DPAD_UP	19		DPAD_DOWN	20
DPAD_LEFT	21		DPAD_RIGHT	22
DPAD_CENTER	23		MENU	82
SEARCH	84		ENTER	66
SPACE	62		DEL	67
TAB	61		ESC	111
APP_SWITCH	187		MEDIA_PLAY_PAUSE	85

Appendix B — minimum viable first session

```
# 1. Get on the car's WiFi, then:
export HU=$(route -n get default | awk '/gateway/{print $2}')
ping -c 4 $HU
nmap -p 5555,22,23,80,8080,7000 $HU

# 2. If 5555 is open:
adb connect $HU:5555
adb devices -l

# 3. If authorized:
adb shell getprop | tee props.txt
adb shell pm list packages -3 | tee third_party_apps.txt
adb shell logcat -d | grep -iE 'gwm|haval|can|car_service' | tee car_logs.txt
adb shell dumpsys car_service | tee car_service.txt
```

References

- [Android ADB documentation](#)
- [dumpsys cheat sheet](#)
- [Android settings database](#)
- [CAN bus basics](#)

Disclaimer: For educational/research purposes on your own vehicle only. You are responsible for any modification. Incorrect changes may brick the head unit, disable vehicle features, or void your warranty.